

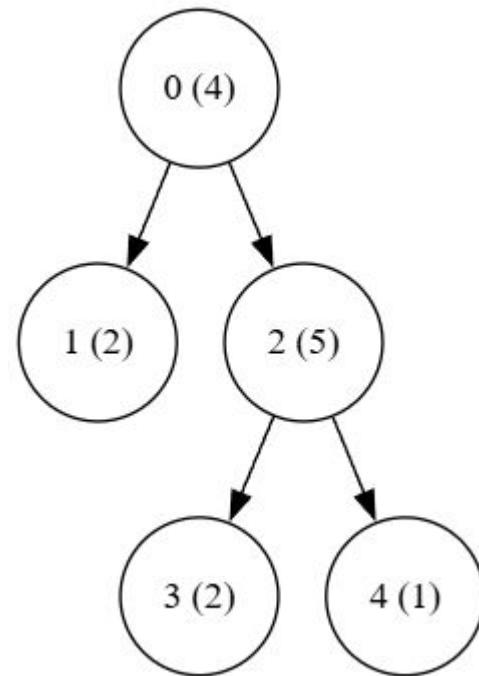
Euler tour + Sparse Table +
BIT

Motivação

- Fonte de material, código, figuras:
<https://usaco.guide/gold/tree-euler?lang=cpp>
- Ideia: linearizar uma árvore para permitir consultas do tipo “nodo mais alto que é filho do 8”, “menor valor de vértice que está na subárvore do vértice 1”, etc. Permite também atualizar valores rapidamente.
- Pasta com material:
https://github.com/sallesviana/INF333-public/tree/main/1-SparseTable_BIT_et_c

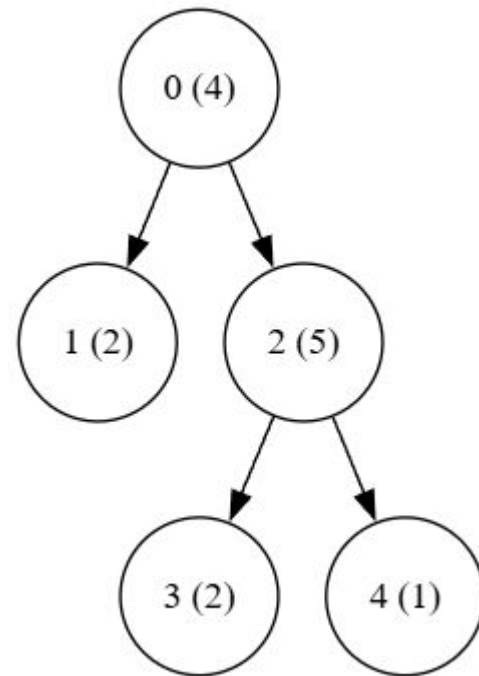
Motivação

- Problema: <https://cses.fi/problemset/task/1137>
 - Árvore de 10^5 vértices ; vértice 1 é a raiz
 - Valores nos vértices (entre parênteses ao lado)
 - Até 10^5 consultas do tipo:
 - 1 v x : troca valor do vértice v para x
 - 2 v : qual a soma dos valores na subárvore de v?
 - Exemplo:
 - consulta 2 2 $\rightarrow$ 8
 - consulta 2 0 $\rightarrow$ 14
 - Ideias de como resolver?



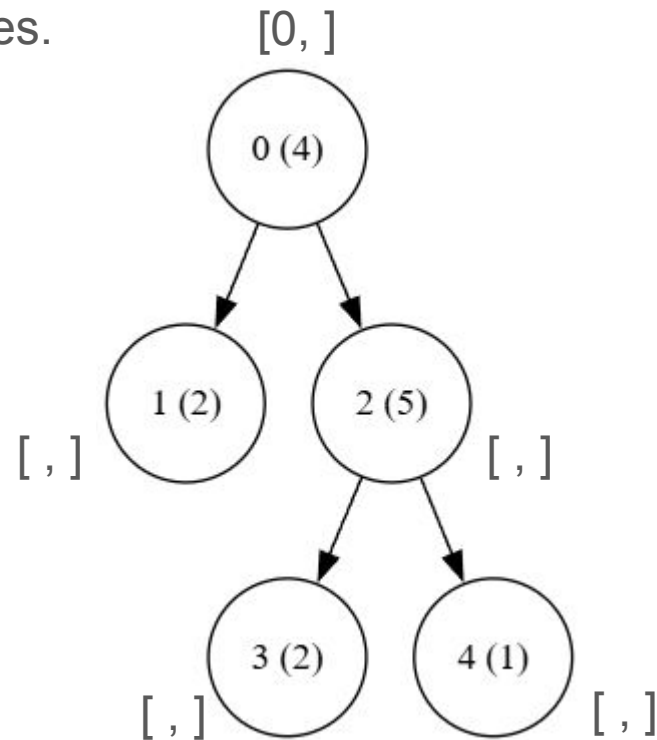
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



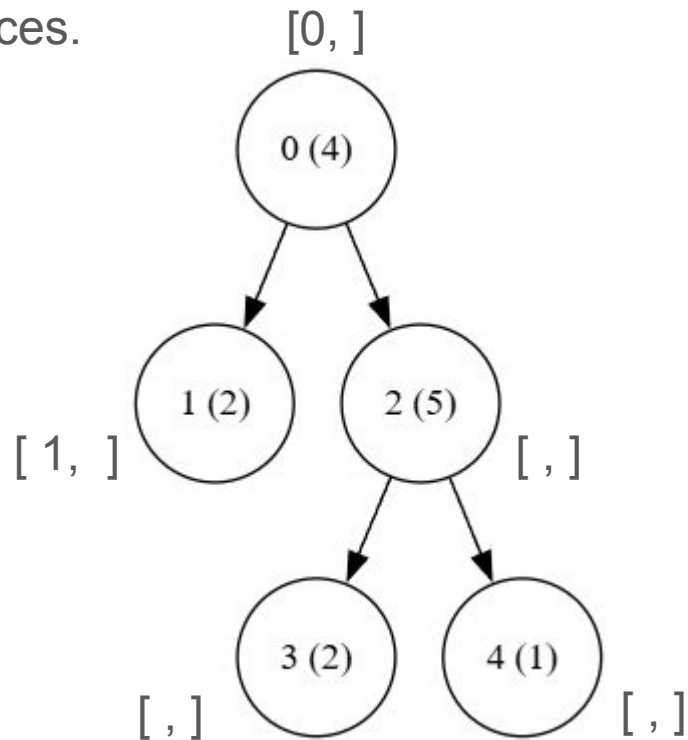
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



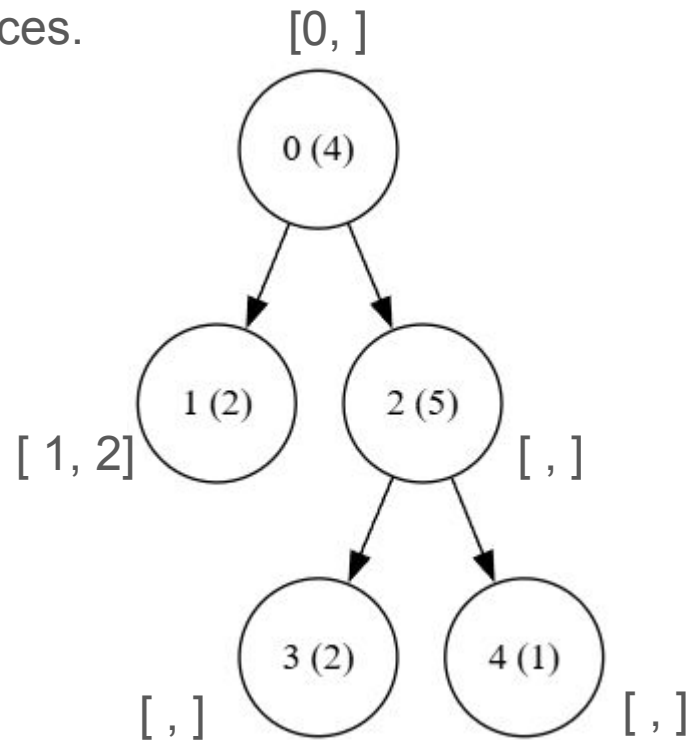
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



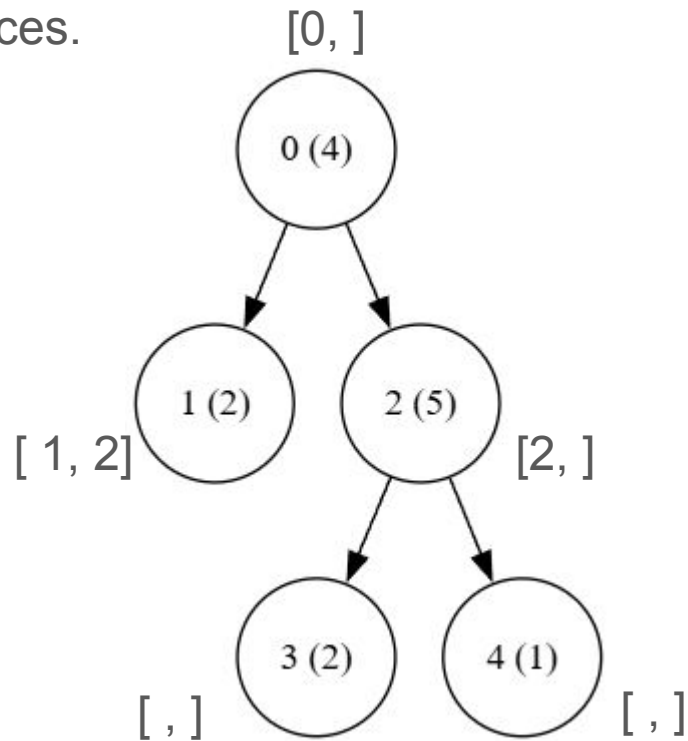
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



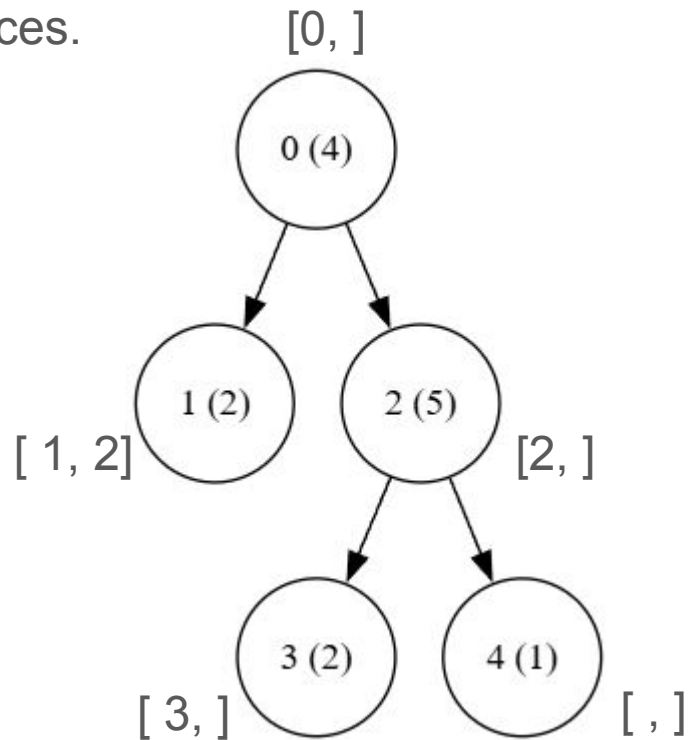
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



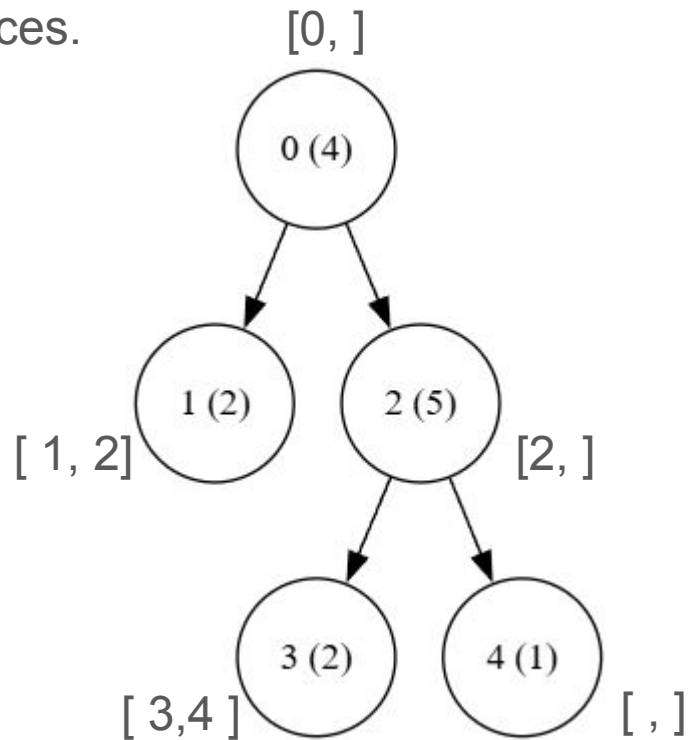
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



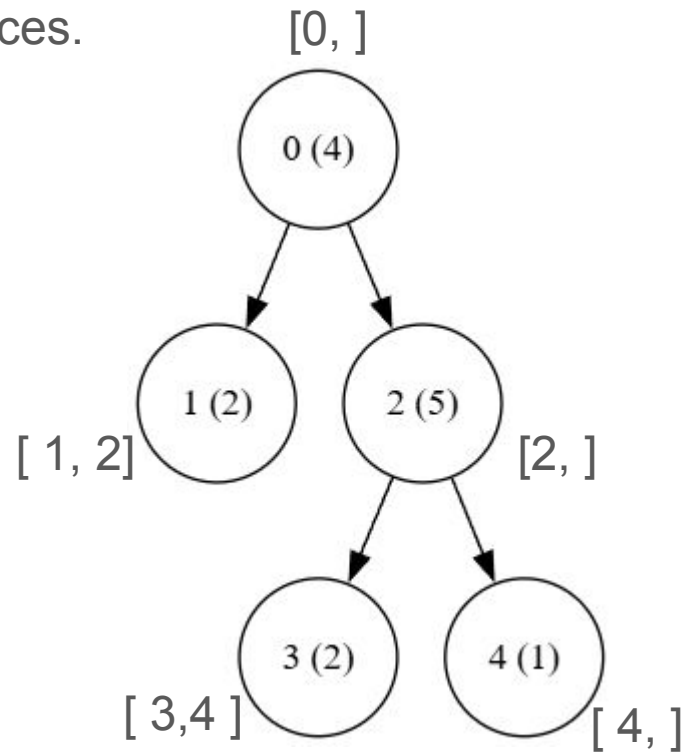
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



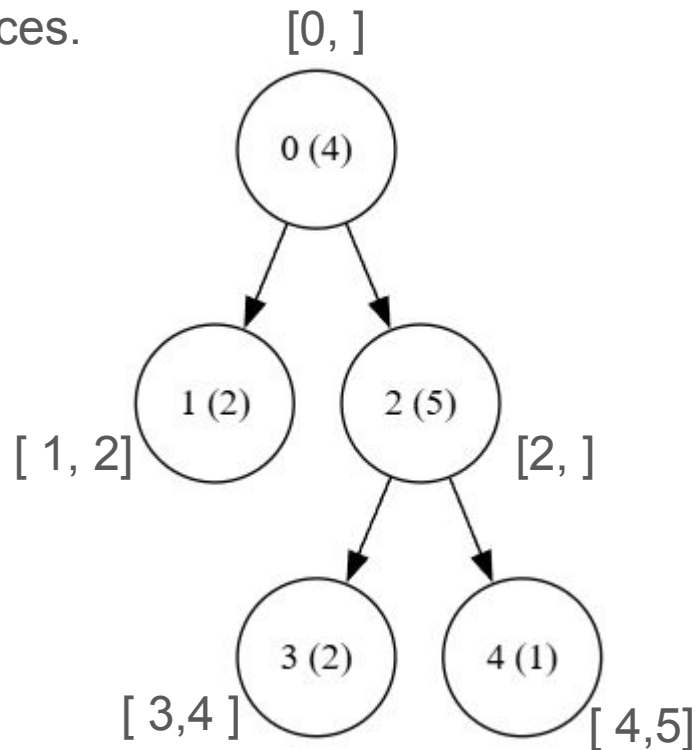
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



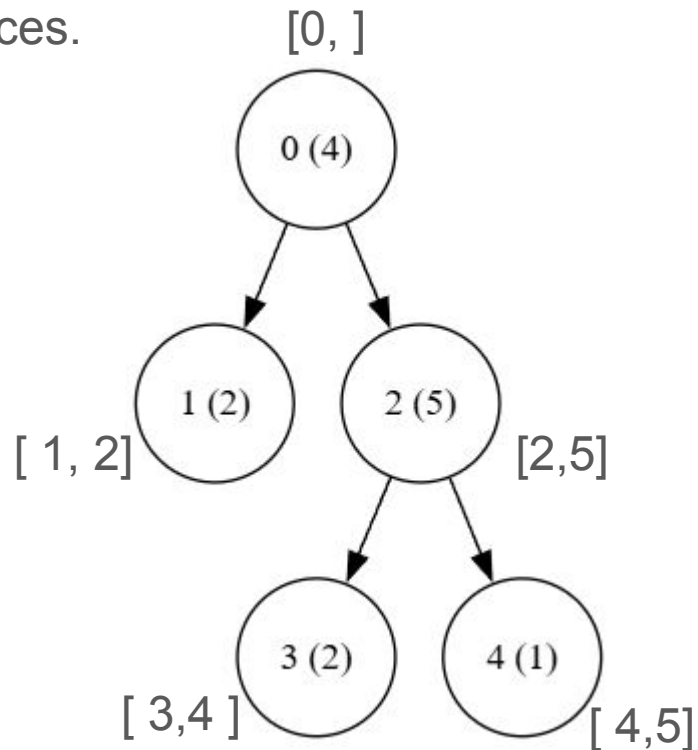
Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



Euler tour

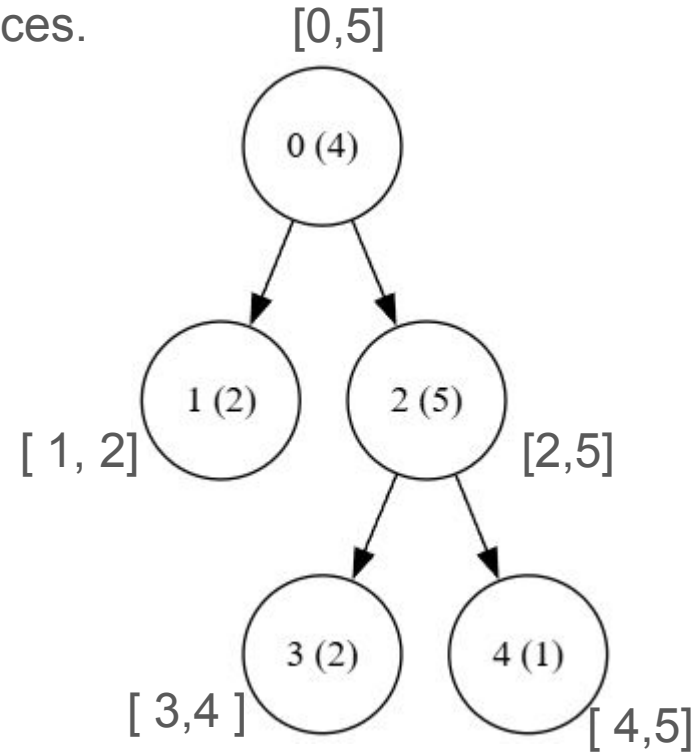
- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v



Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v

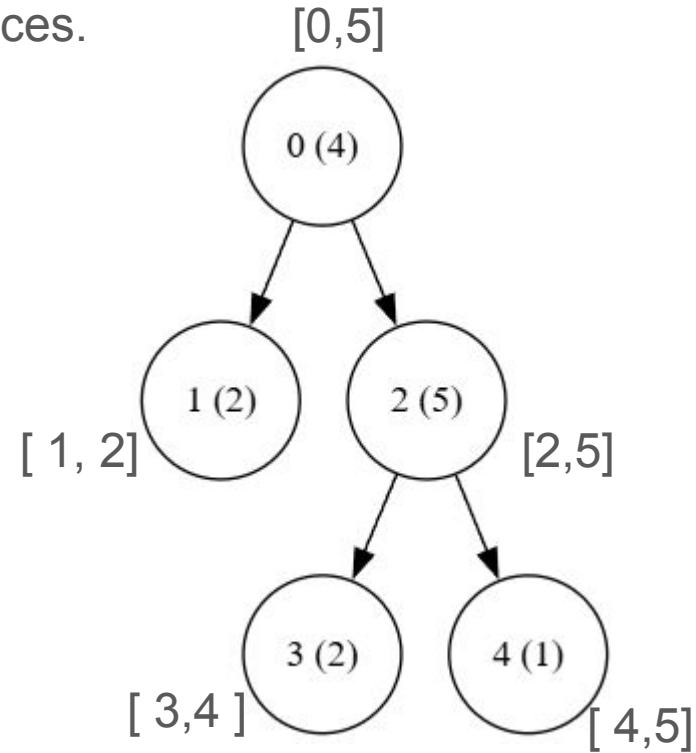
	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5



Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v
- Propriedade importante: ?
 -

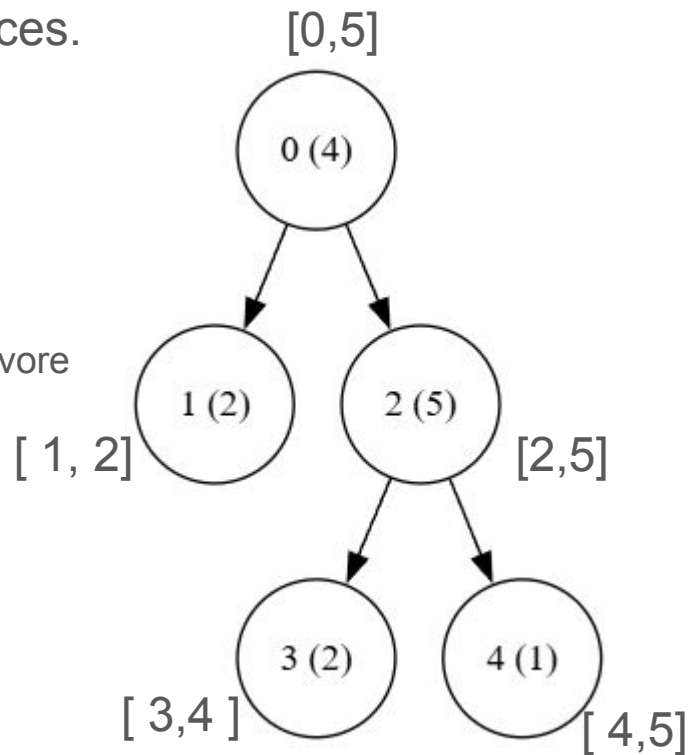
	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5



Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v
- Propriedade importante:
 - $[\text{start}[v], \text{end}[v])$ contém o start de toda a subárvore de v
 - $[2,5] \rightarrow$ contém os starts 2, 3 e 4 (dos 3 elementos da árvore de 2)
 - Qual o tamanho da subárvore de v ?

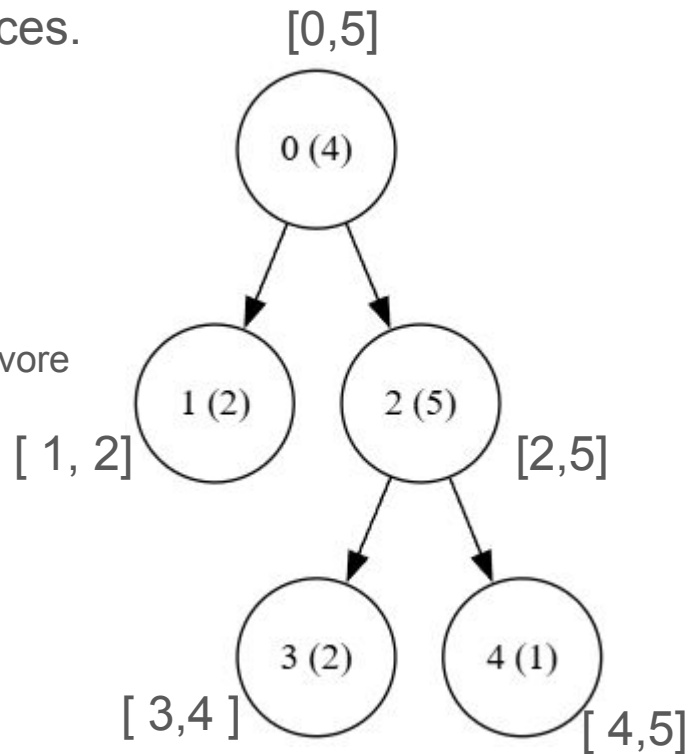
	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5



Euler tour

- DFS que usa um timer para ir marcando os vértices.
 - Calcula dois vetores start, end.
 - $\text{start}[v]$ = momento em que entra em v
 - $\text{end}[v]$ = momento após processar toda subárvore de v
- Propriedade importante:
 - $[\text{start}[v], \text{end}[v])$ contém o start de toda a subárvore de v
 - $[2,5] \rightarrow$ contém os starts 2, 3 e 4 (dos 3 elementos da árvore de 2)
 - Qual o tamanho da subárvore de v ? $\text{end}[v] - \text{start}[v]$

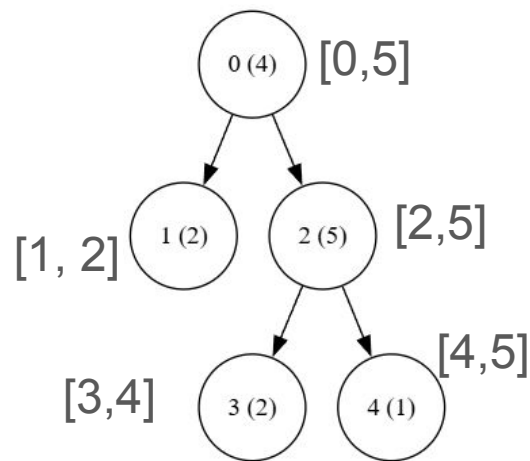
	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5



Euler tour

- Propriedade importante:
 - $[start[v], end[v])$ contém o start de toda a subárvore de v
 - $[2,5] \rightarrow$ contém os starts 2, 3 e 4 (dos 3 elementos da árvore de 2)
 - Qual o tamanho da subárvore de v ? $end[v]-start[v]$
- Soma da subárvore:
 - Cria array com tamanho N com valores.
 - valor de v ficará na posição $start[v]$ (cuidado que $start[v]$ não é sempre $= v$)

	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5

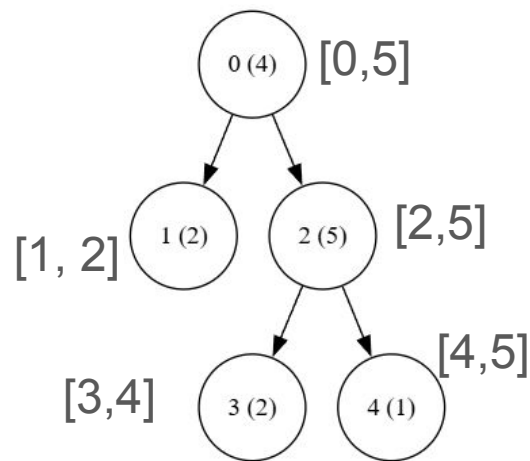


Euler tour

- Propriedade importante:
 - $[start[v], end[v])$ contém o start de toda a subárvore de v
 - $[2,5] \rightarrow$ contém os starts 2, 3 e 4 (dos 3 elementos da árvore de 2)
 - Qual o tamanho da subárvore de v ? $end[v]-start[v]$
- Soma da subárvore:
 - Cria array com tamanho N com valores.
 - valor de v ficará na posição $start[v]$ (cuidado que $start[v]$ não é sempre $= v$)
 - Subárvore de 0 tem start em $[0,5) \rightarrow soma(0) = soma[0,5]$
 - Subárvore de 2 teria $soma(2) =$

	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5

start	0	1	2	3	4
valor	4	2	5	2	1

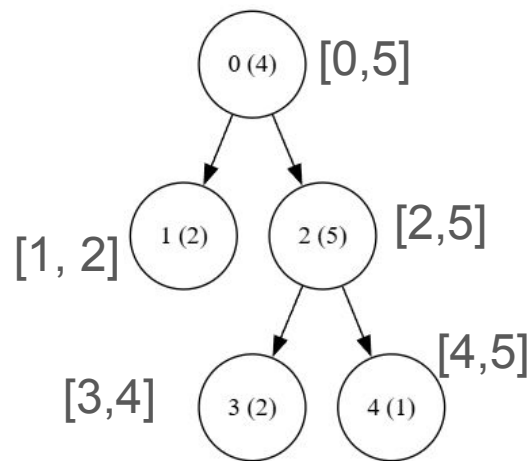


Euler tour

- Propriedade importante:
 - $[start[v], end[v])$ contém o start de toda a subárvore de v
 - $[2,5) \rightarrow$ contém os starts 2, 3 e 4 (dos 3 elementos da árvore de 2)
 - Qual o tamanho da subárvore de v ? $end[v]-start[v]$
- Soma da subárvore:
 - Cria array com tamanho N com valores.
 - valor de v ficará na posição $start[v]$ (cuidado que $start[v]$ não é sempre $= v$)
 - Subárvore de 0 tem start em $[0,5) \rightarrow soma(0) = soma[0,5)$
 - Subárvore de 2 tem start em $[2,5) \rightarrow soma(2) = soma[2,5)$

	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5

start	0	1	2	3	4
valor	4	2	5	2	1

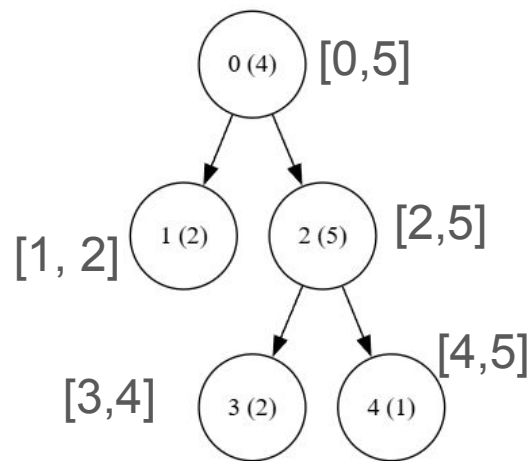


Exercício

- Resolva o problema: <https://cses.fi/problemset/task/1137>
 - Disponível no vjudge da aula de hoje.
- Use a BIT disponível no github da aula para soma de prefixos e atualização.

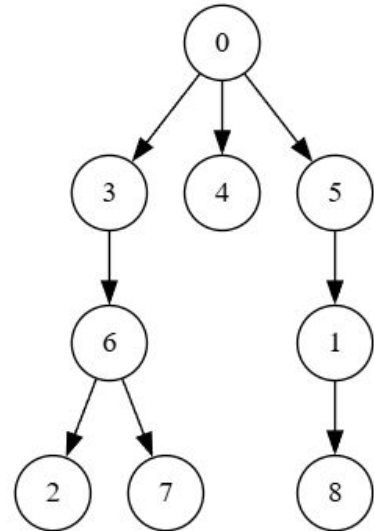
	0	1	2	3	4
start	0	1	2	3	4
end	5	2	5	4	5

start	0	1	2	3	4
valor	4	2	5	2	1



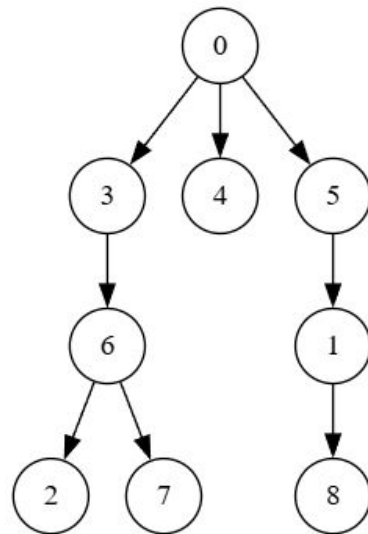
Motivação 2

- Calcular LCA de dois vértices.
 - Como resolver?



Motivação 2

- Calcular LCA de dois vértices.
 - Usar binary/sqrt jump/decomposition para dar “saltos” grandes na árvore
 - Usar Euler tour

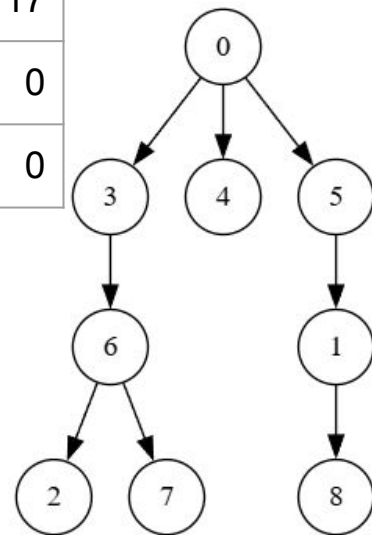


Motivação 2

- Calcular LCA de dois vértices.
 - Usar Euler tour, calculando vetores:
 - tin: tempo no qual vértice aparece pela primeira vez no tour
 - dep: profundidade
 - et: euler tour (tamanho máximo $2n$)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

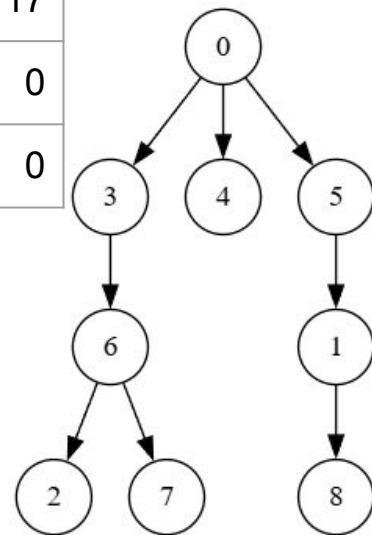


Motivação 2

- Calcular LCA de dois vértices.
 - Usar Euler tour, calculando vetores:
 - tin: tempo no qual vértice aparece pela primeira vez no tour. Exemplo: $\text{tin}[1] = 12$
 - dep: profundidade
 - et: euler tour (tamanho máximo $2n$)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

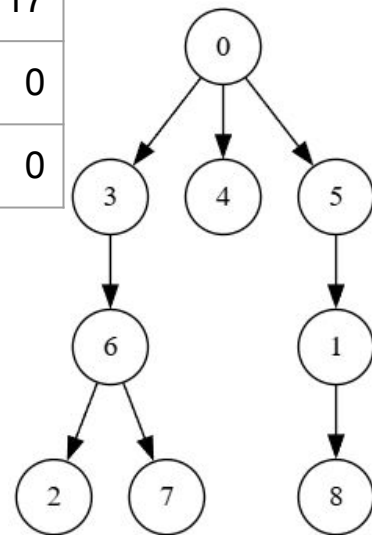


Motivação 2

- Calcular LCA de dois vértices.
 - Usar Euler tour, calculando vetores:
 - tin: tempo no qual vértice aparece pela primeira vez no tour
 - dep: profundidade
 - et: euler tour (tamanho máximo $2n$)
 - $LCA(6,4) = \text{menor número entre tin}[6] \text{ e tin}[4]$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

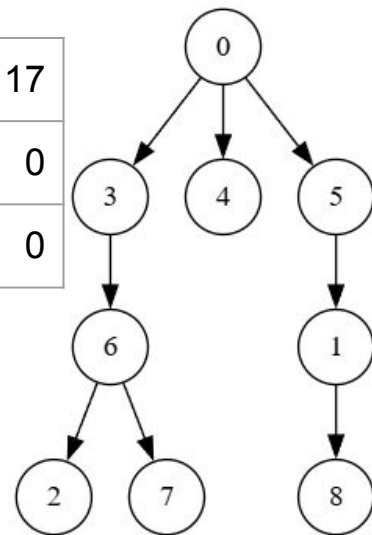


Motivação 2

- Calcular LCA de dois vértices.
 - Usar Euler tour, calculando vetores:
 - $LCA(6,4) = \text{menor número entre } tin[6] \text{ e } tin[4]$
 - Colocar vetor com pares de (dep, et) e achar menor par (menor dep) em $[tin[6], tin[4]]$
 - Como par compara o first $\rightarrow$ achará o vértice de menor profundidade
 - Nesse caso, vai retornar (0,0) . Aí, pegamos o second do par (vértice).
 - Usar estrutura de RMQ para encontrar menor valor no intervalo.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

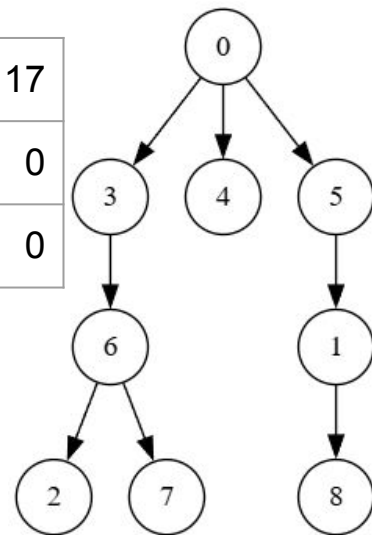


Motivação 2

- RMQ:
 - Seg-tree
 - Tudo $O(\log n)$, em geral
 - Com update
 - Sparse-table:
 - Criação em $O(n \log n)$
 - Consultas MIN em $O(1)$
 - Sem updates

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

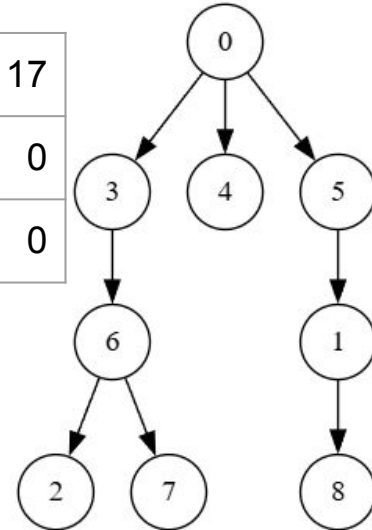


Motivação 2

- Ver exemplo em lca.cpp na pasta da aula (do github)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3

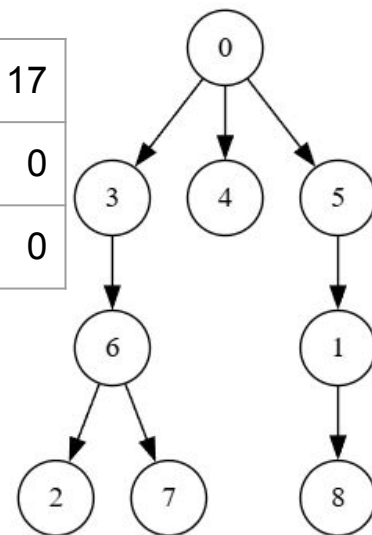


Exercício:

- Fazer exercício (trivial) de sparse table para aprender a usar.

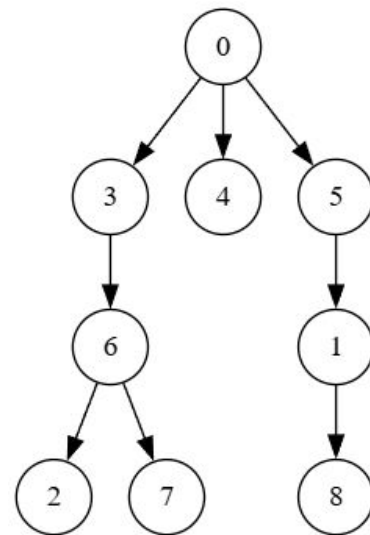
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
dep	0	1	2	3	2	3	2	1	0	1	0	1	2	3	2	1	0	0
et	0	3	6	2	6	7	6	3	0	4	0	5	1	8	1	5	0	0

	0	1	2	3	4	5	6	7	8
tin	0	12	3	1	9	11	2	5	13
dep	0	2	3	1	1	1	2	3	3



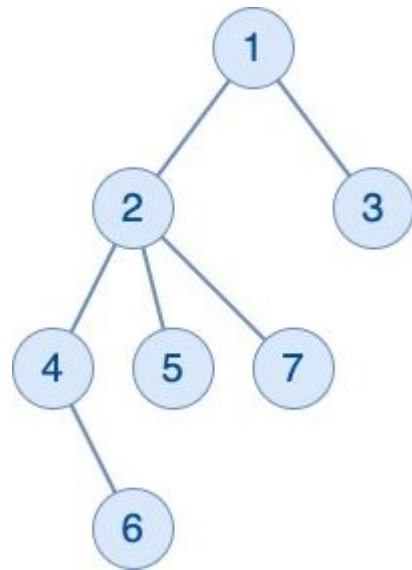
Exercício de Euler tour:

- Fazer exercício de Euler tour:
 - https://atcoder.jp/contests/abc202/tasks/abc202_e?lang=en
- Dicas nos próximos slides



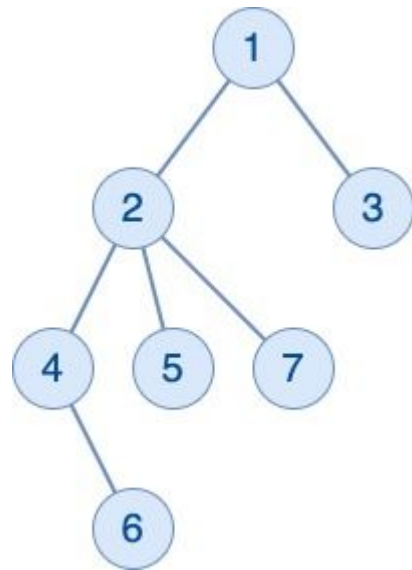
Exercício de Euler tour:

- Entrada: árvore e consultas
- Consulta: vért, dist = quantos vértices há com distância 2 da raiz e com ancestral vert?
 - Ex: 2 2 $\rightarrow$ há 3 vértices (4,5,7) com distância 2 da raiz e que tem 2 como ancestral.
 - 7 2 $\rightarrow$ há 1 vértice (7) com distância 2 da raiz e com 7 de ancestral (inclusive)



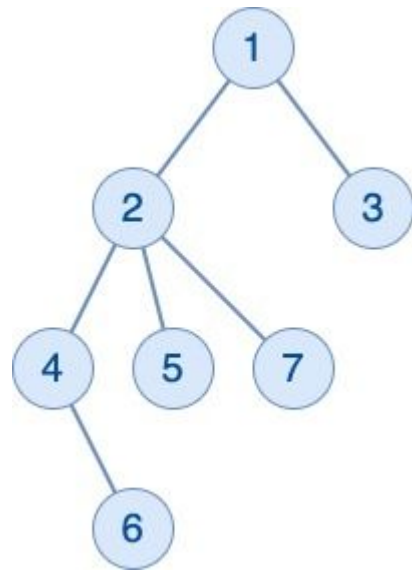
Exercício de Euler tour:

- Entrada: árvore e consultas
- Consulta: vért, dist = quantos vértices há com distância 2 da raiz e com ancestral vert?
 - Ex: 2 2 $\rightarrow$ há 3 vértices (4,5,7) com distância 2 da raiz e que tem 2 como ancestral.
 - 7 2 $\rightarrow$ há 1 vértice (7) com distância 2 da raiz e com 7 de ancestral (inclusive)
 - Em outras palavras, quantos vértices da subárvore de vert estão em distância dist da raiz?



Exercício de Euler tour:

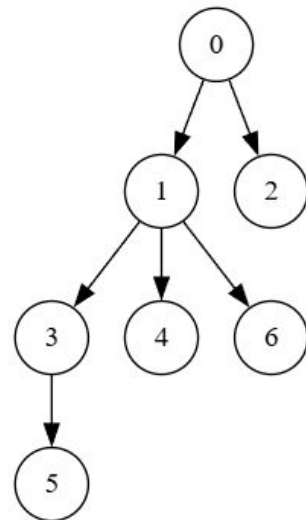
- Entrada: árvore e consultas
- Consulta: vért, dist = quantos vértices há com distância 2 da raiz e com ancestral vert?
 - Ex: 2 2 $\rightarrow$ há 3 vértices (4,5,7) com distância 2 da raiz e que tem 2 como ancestral.
 - 7 2 $\rightarrow$ há 1 vértice (7) com distância 2 da raiz e com 7 de ancestral (inclusive)
 - Em outras palavras, quantos vértices da subárvore de vert estão em distância dist da raiz?
- Ideias? Dica: Euler tour (quem diria...)



Exercício de Euler tour:

- Entrada: árvore e consultas
- Consulta: vért, dist = quantos vértices há com distância 2 da raiz e com ancestral vert?
 - Em outras palavras, quantos vértices da subárvore de vert estão em distância dist da raiz?
- Ideias? Dica: Euler tour (quem diria...)
 - Em uma Euler tour, os vértices que tiverem entrada (tin) entre o momento da entrada (tin) do 1 e da saída (tout) do 1, são sua subárvore.
 - Precisamos achar quantos vértices desse intervalo possuem distância dist..

	0	1	2	3	4	5	6
tin	0	1	6	2	4	3	5
tout	7	6	7	4	5	4	6
dep	0	1	1	2	2	3	2

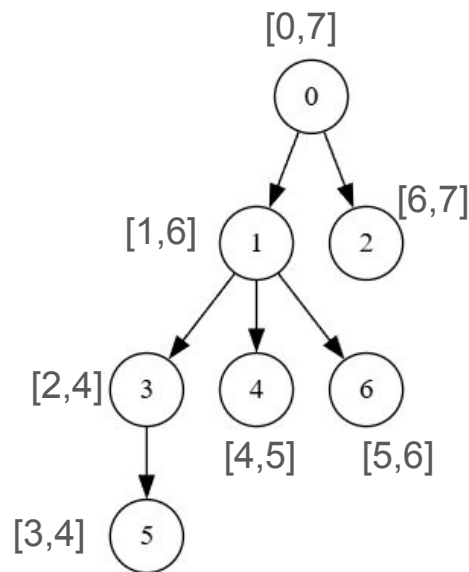


* árvore refeita contando de 0

Exercício de Euler tour:

- Ideias? Dica: Euler tour (quem diria...)
 - Em uma Euler tour, os vértices que tiverem entrada (tin) entre o momento da entrada (tin) do 1 e da saída (tout) do 1, são sua subárvore.
 - Vértice 1: $[tin, tout) = [1, 6) \rightarrow$ vértices com tin entre 1 e 6) estão na subárvore de 1
 - Precisamos contar quantos desses vértices possuem $dep = 2$

	0	1	2	3	4	5	6
tin	0	1	6	2	4	3	5
tout	7	6	7	4	5	4	6
dep	0	1	1	2	2	3	2

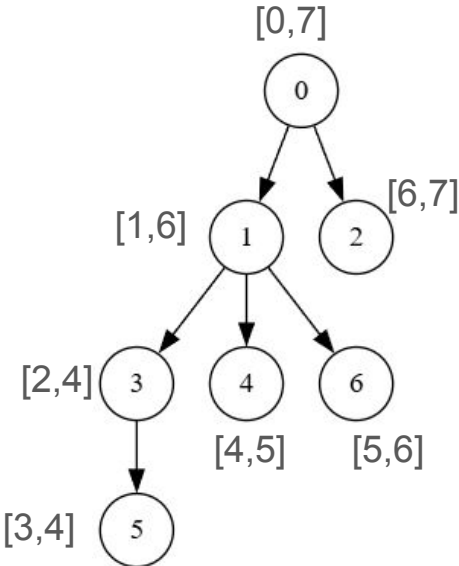


Exercício de Euler tour:

- Ideias? Dica: Euler tour (quem diria...)
 - Em uma Euler tour, os vértices que tiverem entrada (tin) entre o momento da entrada (tin) do 1 e da saída (tout) do 1, são sua subárvore.
 - Vértice 1: $[tin, tout) = [1, 6) \rightarrow$ vértices com tin entre 1 e 6) estão na subárvore de 1
 - Precisamos contar quantos desses vértices possuem dep = 2
 - Uma solução: guardar para cada profundidade os tins dos vértices nela.
 - Consulta para dep 2, vert 1: ver quantos números há em $[1, 6)$ em dep 2
 - Consulta para dep 1, vert 1: ?

dep 0	0		
dep 1	1	6	
dep 2	2	4	5
dep 3	3		

	0	1	2	3	4	5	6
tin	0	1	6	2	4	3	5
tout	7	6	7	4	5	4	6
dep	0	1	1	2	2	3	2



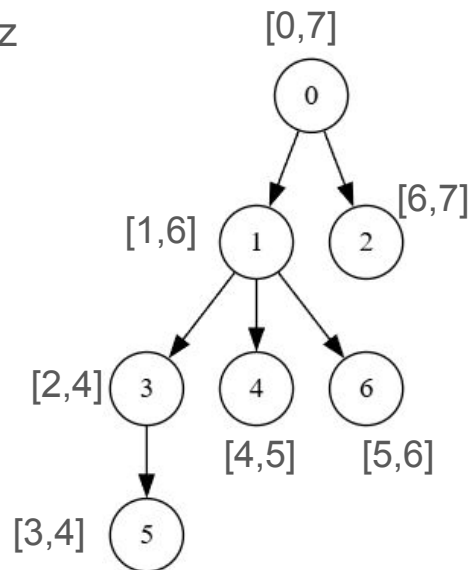
* árvore refeita contando de 0

Exercício de Euler tour:

- Ideias? Dica: Euler tour (quem diria...)
 - Em uma Euler tour, os vértices que tiverem entrada (tin) entre o momento da entrada (tin) do 1 e da saída (tout) do 1, são sua subárvore.
 - Vértice 1: $[tin, tout) = [1, 6) \rightarrow$ vértices com tin entre 1 e 6) estão na subárvore de 1
 - Precisamos contar quantos desses vértices possuem dep = 2
 - Uma solução: guardar para cada profundidade os tins dos vértices nela.
 - Consulta para dep 2, vert 1: ver quantos números há em $[1, 6)$ em dep 2
 - Consulta para dep 1, vert 1: números em $[1, 6)$ em dep 1
- Dica adicional: adaptei o código de LCA para calcular esses arrays e fiz as contagens usando lower_bound para busca binária.

dep 0	0		
dep 1	1	6	
dep 2	2	4	5
dep 3	3		

	0	1	2	3	4	5	6
tin	0	1	6	2	4	3	5
tout	7	6	7	4	5	4	6
dep	0	1	1	2	2	3	2



Outro problema

- Uno (mineira 2024)
 - <https://codeforces.com/group/YgJmumGtHD/contest/528947/problem/L>